B V RAJU INSTITUTE OF TECHNOLOGY
Vishnupur, Narsapur, Medak, Telangana — 502313
Department of Computer Science and Engineering (Data Science)

TECHNICAL HANDOVER DOCUMENT

Geometric Linguistic Quantifier (GLQ)

A Semantic Anchor Approach to Measuring Code-Mixed Density in Hindi–Telugu–English Corpora

Author	Vukkem Bhuvaneshwar
Roll Number	22211A67C3
Guide	Mr. Krutibash Nayak, Assistant Professor, CSE (DS)
Programme	Bachelor of Technology — Computer Science and Engineering (Data Science)
Project	Major Project Phase-2
Academic Year	2025 – 2026
Document Purpose	This document is a complete technical handover guide intended for junior developers who will continue

Table of Contents

CHAPTER 1

Project Overview and Background

1.1 What is the Geometric Linguistic Quantifier?

The Geometric Linguistic Quantifier (GLQ) is an NLP system designed to measure, in real time, the linguistic composition of sentences that mix Hindi, Telugu, and English. Rather than assigning a single language label to a sentence, GLQ outputs a percentage breakdown — for example: *Hindi 53%, English 26%, Telugu 21%*. This approach is termed **linguistic density quantification** and stands in contrast to conventional Language Identification (LID) systems, which treat language assignment as a discrete classification problem.

The system was developed as a B.Tech Final Year Major Project (Phase-2) in the Department of Computer Science and Engineering (Data Science) at B V Raju Institute of Technology, Narsapur, for the academic year 2025–2026.

1.2 The Problem Being Solved

In contemporary Indian digital communication — particularly on social media, messaging platforms, and professional chat tools — speakers routinely blend Hindi, Telugu, and English within a single utterance. This phenomenon, known as **code-mixing**, produces hybrid sentences such as those shown below.

Register	Example Sentence
Hinglish	Mera computer charge nahi ho raha, network bhi bahut slow hai.
Tenglish	Meeting ki preparation cheyyadaniki documents ready ga undi.
Trilingual	Server down ho gaya, database bhi crash kar gaya, idi fix cheyyali.

Existing tools — Google CLD3, Meta fastText, langid.py — assign a single language label per token or per sentence. They cannot produce a statement such as 'this sentence is 53% Hindi'. They also fail on loanwords such as *charge*, *network*, or *crash* that appear across all three language vocabularies without any syntactic distinction. GLQ addresses both shortcomings.

1.3 The Solution in Brief

GLQ treats each language as a **direction in a 384-dimensional vector space**. Any input sentence is encoded as a point in that same space using a pre-trained multilingual deep-learning model (Sentence-BERT). The angular distance between the input point and each language's representative direction determines the percentage attributed to that language. This geometric approach entirely replaces vocabulary lookups and rule-based classifiers.

1.4 Project Scope

The project covers three languages (Hindi, Telugu, English), operates at the sentence level, and is deployed as a Streamlit web application accessible via a standard web browser. The system does not require a GPU at runtime; GPU hardware is only used during the one-time offline anchor generation step.

CHAPTER 2

Core Concepts

This chapter explains the four technical ideas that underpin the GLQ system. A thorough understanding of these concepts is essential before reading the code.

2.1 Sentence Embeddings

The system uses a pre-trained multilingual transformer model — **paraphrase-multilingual-MiniLM-L12-v2**, available via Hugging Face — to convert any text string into a fixed-length numerical vector of 384 dimensions. This process is called **embedding**. The model was trained on parallel corpora in over 50 languages, so sentences with similar meaning in different languages are mapped to nearby points in the 384-dimensional space. Sentences from the same language, regardless of their topic, cluster together in distinct regions of this space.

2.2 Semantic Anchors

A Semantic Anchor is a unit vector in the 384-dimensional embedding space that represents the centroid of a cluster of sentences drawn from a specific language and contextual register. The system uses nine anchors in total: four for Hindi (narrative, formal, casual, and technical registers), four for Telugu (same registers), and one global anchor for English.

Formally, for language L and register C, the anchor is defined as:

```
B(L, C) = unit_normalise( (1/N) * sum( embed(sentence_i) ) )

where: N = number of representative sentences
embed() = SentenceTransformer encoder
unit_norm = divide by L2 magnitude (projects onto unit hypersphere)
```

Equation 1: Semantic Anchor Construction

Anchor	Language	Register	Corpus Source	Approx. Sentences
A1	Hindi	Narrative	IndicCorp	70,000
A2	Hindi	Formal	IndicCorp	65,000
A3	Hindi	Casual	IndicCorp	80,000
A4	Hindi	Technical	IndicCorp	70,000
A5	Telugu	Narrative	IndicCorp	72,000
A6	Telugu	Formal	IndicCorp	68,000
A7	Telugu	Casual	IndicCorp	75,000
A8	Telugu	Technical	IndicCorp	67,784

A9	English	Global	Wikitext-103	25,000

Table 1: Nine Semantic Anchors used by GLQ

2.3 Contextual Winnowing

A naive approach would compute similarity between the input sentence and a single anchor per language. However, a technical sentence written in Telugu will score high English similarity because shared technical vocabulary — *server*, *database*, *API* — occupies overlapping embedding regions. The Contextual Winnowing mechanism prevents this by maintaining four anchors per language (one per register) and selecting only the maximum similarity score:

```
S_Hindi = max( sim(v, A1), sim(v, A2), sim(v, A3), sim(v, A4) )
S_Telugu = max( sim(v, A5), sim(v, A6), sim(v, A7), sim(v, A8) )
S_English = sim(v, A9)

where sim(a, b) = dot_product(a, b) # both vectors are unit-normalised,
# so dot product == cosine similarity
```

Equation 2: Contextual Winnowing

A technical Telugu sentence therefore matches A8 (Telugu-Technical) with high similarity, rather than inflating the English score. This is the principal innovation that distinguishes GLQ from token-counting approaches.

2.4 Temperature-Controlled Softmax Calibration

The three per-language scores produced by winnowing are converted to percentages using a Softmax function parameterised by a temperature value τ:

```
P(L_i) = exp( S_i / tau ) / sum_j( exp( S_j / tau ) )

tau = 0.10 --> hard classification (dominant language ~ 85-90%)
tau = 0.40 --> soft density estimate (DEFAULT, recommended for analysis)
tau = 1.00 --> near-uniform distribution (exploratory use only)
```

Equation 3: Temperature-Controlled Softmax

At low τ values the system behaves as a hard language identifier; at higher τ values the distribution reflects the genuine multilingual composition of the utterance. The default of $\tau = 0.4$ was selected empirically using the 500-sentence evaluation set.

CHAPTER 3

System Architecture

3.1 Two-Phase Design

The GLQ system is structured as two entirely separate pipelines. Understanding this separation is fundamental before working on the codebase.

Phase	Script	Trigger	Output
Offline (one-time)	generate_anchors.py	Run once before first deployment; re-run when adding a language	Nine anchor files in data/baselines/
Online (per query)	web_quantifier.py + infere	Triggered automatically by every user text submission in the browser	Language probability dictionary and Alta

Table 2: Two-Phase System Design

3.2 Offline Baseline Generation Pipeline

The offline pipeline is responsible for constructing the nine semantic anchor vectors from raw corpus data. It consists of the following sequential stages:

Stage 1: Corpus Loading

The IndicCorp JSON file is parsed line by line to extract Hindi and Telugu sentences. English sentences are sampled from Wikitext-103. The combined corpus contains approximately 1,152,784 sentences.

Stage 2: Batch Vectorisation

Sentences are processed in batches of 64 by the SentenceTransformer encoder (paraphrase-multilingual-MiniLM-L12-v2) with mean-pooling over the final hidden layer. Each sentence yields a 384-dimensional float32 array. The parameter normalize_embeddings=True is passed to ensure all output vectors are L2-normalised onto the unit hypersphere.

Stage 3: K-Means Clustering

scikit-learn's KMeans algorithm (k=4, init='k-means++', random_state=42) is applied separately to the Hindi embedding matrix and the Telugu embedding matrix. The four cluster centroids per language correspond loosely to the narrative, formal, casual, and technical registers. English uses a single global mean vector; no clustering is applied.

Stage 4: Centroid Normalisation

Each of the nine centroids is divided by its L2 magnitude to project it onto the surface of the unit hypersphere. This normalisation ensures that cosine similarity at runtime equals the dot product, eliminating the need for a separate normalisation step during inference.

Stage 5: Serialisation

Each normalised anchor vector is saved to disk as a binary PyTorch tensor file (.pt) using torch.save(). These files are small (a few kilobytes each) and load in milliseconds at application startup.

3.3 Online Inference Pipeline

The online pipeline runs for every user query submitted through the Streamlit interface. It is designed to be stateless and operates entirely on pre-computed anchor vectors:

Stage 1: Anchor Loading (once at startup)

All nine .pt anchor vectors are loaded into memory using torch.load() when the Streamlit application starts. They are cached in module-level variables so that subsequent queries do not incur disk reads.

Stage 2: Input Embedding

The user's text string is encoded by the same SentenceTransformer model used during offline generation. The pooling configuration and normalisation must match exactly: normalize_embeddings=True is mandatory.

Stage 3: Cosine Similarity Computation

The dot product is computed between the unit-normalised input vector and each of the nine unit-normalised anchor vectors, yielding nine similarity scores in the range [-1, 1].

Stage 4: Contextual Winnowing

For each language, the maximum similarity score across its register-specific anchors is selected, reducing nine scores to three per-language scores.

Stage 5: Softmax Calibration

The three scores are divided by τ and passed through PyTorch's F.softmax function, producing a probability distribution that is multiplied by 100 to yield percentages.

Stage 6: Visualisation

Streamlit renders a horizontal bar chart (Altair), a heatmap of raw similarity scores against all nine anchors, and the numeric output table.

CHAPTER 4

Project Structure and File Reference

4.1 Directory Layout

```
GLQ Final/
|-- web_quantifier.py Main Streamlit application (entry point)
|-- generate_anchors.py Offline anchor generation script
|-- inference.py Core inference logic (imported by web_quantifier)
|-- requirements.txt Python dependency list
|-- Roadmap.txt Original project development roadmap
|-- evaluation/
| `-- annotated_500.csv 500-sentence hand-annotated evaluation dataset
`-- data/
`-- baselines/
|-- hindi_narrative.pt
|-- hindi_formal.pt
|-- hindi_casual.pt
|-- hindi_technical.pt
|-- telugu_narrative.pt
|-- telugu_formal.pt
|-- telugu_casual.pt
|-- telugu_technical.pt
`-- english_global.pt
```

4.2 File Descriptions

File	Purpose	When to Modify
web_quantifier.py	Streamlit UI: text input, temperature slider, bar chart, similarity heatmap, t-SNE projection visualisation [illegible]	When changing the UI, adding [illegible] examples, or [illegible]
generate_anchors.py	Offline pipeline: loads corpora, batch-encodes sentences, runs K-Means, [illegible] centroids, serialis[illegible]	When adding new languages, changing k, or [illegible]
inference.py	Pure algorithm: loads .pt files at startup, embeds input, computes cosine similarity, [illegible]	When modifying the scoring algorithm, contextual [illegible]
data/baselines/*.pt	Pre-computed anchor tensors. One file per language-register pair (nine total). Binary PyTorch float32 te[illegible]	Never edit manually. Always regenerate via gene[illegible]
requirements.txt	Pinned Python dependency versions for reproducible environments.	When adding a new library to the project.

Table 3: File Reference

CHAPTER 5

Environment Setup

5.1 System Prerequisites

Requirement	Specification	Notes
Python	3.10 or later	3.11 and 3.12 are also supported. Python 3.8 is not.
pip	Latest version	Upgrade with: python -m pip install --upgrade pip
NVIDIA GPU + CUDA	CUDA 11.8+ (optional)	Required only for baseline generation. Runtime inference runs on CPU.
RAM	8 GB minimum	16 GB or more is recommended for baseline generation.
Disk space	Approximately 10 GB	Model weights ~400 MB; corpora ~3 GB (if regenerating baselines); anchor files are
Internet connection	Required once	Needed to download the SentenceTransformer model from Hugging Face on first ru

Table 4: System Prerequisites

5.2 Installation Procedure

The following commands should be executed in a terminal from the project root directory with Python 3.10 or later available on the system path.

```
# 1. Navigate to the project folder
cd 'GLQ Final'

# 2. Create a virtual environment (recommended)
python -m venv venv
source venv/bin/activate # Linux / macOS
venv\Scripts\activate # Windows (Command Prompt or PowerShell)

# 3. Install all required libraries
pip install streamlit torch sentence-transformers pandas altair numpy scikit-learn

# 4. Verify the installation
python -c "from sentence_transformers import SentenceTransformer; print('OK')"

# The model (~400 MB) is downloaded automatically from Hugging Face on first use.
# Model identifier: paraphrase-multilingual-MiniLM-L12-v2
```

> ***Important:*** *If the deployment machine has no internet access, pre-download the model on a connected machine by running: python -c "from sentence_transformers import SentenceTransformer; SentenceTransformer('paraphrase-multilingual-MiniLM-L12-v2')" and copy the resulting cache directory (~/.cache/huggingface/) to the target machine.*

5.3 Corpus Datasets (Required for Baseline Regeneration Only)

The pre-built anchor files in data/baselines/ are ready for immediate use. The datasets listed below are only needed if you intend to regenerate the anchor files from scratch or extend the system to additional languages.

Dataset	Source URL	Used For	Size
IndicCorp	ai4bharat.org / HuggingFace: ai4bharat/IndicCorpus	Hindi and Telugu sentence corpora	~3 GB
Wikitext-103	huggingface.co/datasets/wikitext	English anchor construction	~500 MB
Annotated Set	Included in repository: evaluation/annotated_500.csv	500 sentences for evaluation only	<1 MB

Table 5: Required Datasets

CHAPTER 6

Generating the Baseline Anchors

> ***Note:*** *This chapter is only relevant if the .pt files in data/baselines/ are missing, or if you are adding a new language to the system. If the pre-built files are already present, proceed directly to Chapter 7.*

6.1 Executing the Generator

With the virtual environment active and the corpus files in place, execute the following command from the project root:

```
python generate_anchors.py

# Expected console output (abridged):
# Loading IndicCorp Hindi ... 1,127,784 sentences found
# Encoding Hindi sentences (batch_size=64) ... [progress bar]
# Running K-Means (k=4) on Hindi embeddings ...
# Saving hindi_narrative.pt ... OK
# Saving hindi_formal.pt ... OK
# Saving hindi_casual.pt ... OK
# Saving hindi_technical.pt ... OK
# [repeated for Telugu and English]
# All 9 anchors saved to data/baselines/

# Approximate run times:
# NVIDIA T4 GPU (Google Colab): approximately 25 minutes
# CPU-only (modern laptop): approximately 2-3 hours
```

6.2 Internal Logic of the Generation Script

The following code excerpt illustrates the core logic of generate_anchors.py for one language. The same pattern is repeated for each language.

```
from sentence_transformers import SentenceTransformer
from sklearn.cluster import KMeans
import numpy as np, torch

model = SentenceTransformer('paraphrase-multilingual-MiniLM-L12-v2')

def generate_anchors(sentences, n_clusters=4, save_prefix='data/baselines/lang'):
# Stage 1: Encode all sentences to unit vectors
embeddings = model.encode(sentences, batch_size=64,
normalize_embeddings=True,
show_progress_bar=True)

# Stage 2: K-Means clustering with k-means++ initialisation
km = KMeans(n_clusters=n_clusters, init='k-means++',
max_iter=300, random_state=42)
km.fit(embeddings)

# Stage 3: Normalise each centroid and serialise
for i, centroid in enumerate(km.cluster_centers_):
anchor = centroid / np.linalg.norm(centroid)
torch.save(torch.tensor(anchor, dtype=torch.float32),
f'{save_prefix}_cluster{i}.pt')
```

6.3 Extending the System to a New Language

To add support for an additional language such as Bengali or Tamil, follow the steps below. Each step references the files that must be modified.

- Obtain a monolingual sentence corpus for the new language. IndicCorp (AI4Bharat) provides data for Bengali, Tamil, Marathi, Kannada, and Gujarati.
- Add a new section in generate_anchors.py that loads, batch-encodes, and K-Means clusters the new language's sentences, following the Hindi/Telugu pattern.
- Add the resulting .pt files (four register anchors, or one global anchor) to data/baselines/.
- In inference.py, add the new language key to the ANCHORS dictionary and ensure it is included in the contextual winnowing loop.
- Update the Streamlit UI in web_quantifier.py to display the new language column in the output table and bar chart.

CHAPTER 7

Running the Web Application

7.1 Starting the Application Server

```
# Activate the virtual environment
source venv/bin/activate # Linux / macOS
venv\Scripts\activate # Windows

# Launch the Streamlit server from the project root
streamlit run web_quantifier.py

# The browser opens automatically at: http://localhost:8501
# To stop the server, press Ctrl+C in the terminal.
```

7.2 Interface Components

Component	Location	Description
Text Input Area	Main panel	Accepts any code-mixed Hindi/Telugu/English text. Submit by pressing Enter or clicking
Temperature Slider (τ)	Sidebar	Adjusts output sharpness. Range 0.05–1.0, default 0.40. Updates the result in real time.
Language Percentages	Main panel	Primary output: three percentage values with a horizontal bar chart (Altair).
Similarity Heatmap	Main panel	Raw cosine similarity scores against all nine anchors. Useful for debugging unexpected
Example Sentence Loader	Sidebar	Pre-loaded Hinglish and Tenglish sentences for demonstration purposes.
t-SNE Projection	Bottom / sidebar	Static 2-D projection of the 384-D embedding space showing cluster separation.

Table 6: Streamlit Interface Components

7.3 Common Errors and Remedies

Error	Likely Cause	Remedy
ModuleNotFoundError: sentence_transformers	Library not installed	pip install sentence-transformers
FileNotFoundError: data/baselines/hindi_anchors.pt	Anchor files missing from disk	Run generate_anchors.py, or copy the backup .pt files into da
OSError: Address already in use (port 8501)	Another Streamlit process running	Use: streamlit run web_quantifier.py --server.port 8502
RuntimeError: CUDA out of memory	Insufficient GPU VRAM	Reduce batch_size in generate_anchors.py from 64 to 16.
UnicodeDecodeError reading corpus	Corpus file not in UTF-8	Add errors='ignore' to the open() call in the loading function.

Table 7: Common Errors and Remedies

CHAPTER 8

Algorithms in Detail

8.1 Baseline Generation

The following is a complete, annotated implementation of the baseline generation function as used in generate_anchors.py.

```
from sentence_transformers import SentenceTransformer
from sklearn.cluster import KMeans
import numpy as np, torch

model = SentenceTransformer('paraphrase-multilingual-MiniLM-L12-v2')

def generate_anchors(sentences, n_clusters=4, save_prefix='data/baselines/lang'):
"""
sentences : list of str — raw corpus sentences for one language
n_clusters : int — number of register clusters (default 4)
save_prefix : str — path prefix for output .pt files
"""
# Encode all sentences; normalize_embeddings=True projects onto unit sphere
embeddings = model.encode(sentences, batch_size=64,
normalize_embeddings=True,
show_progress_bar=True)

# K-Means with k-means++ seeding for deterministic convergence
km = KMeans(n_clusters=n_clusters, init='k-means++',
max_iter=300, random_state=42)
km.fit(embeddings)

# Project each centroid onto the unit hypersphere and save
for i, centroid in enumerate(km.cluster_centers_):
anchor = centroid / np.linalg.norm(centroid)
torch.save(
torch.tensor(anchor, dtype=torch.float32),
f'{save_prefix}_cluster{i}.pt'
)
print(f'Saved {save_prefix}_cluster{i}.pt')
```

8.2 Runtime Inference

The following is a complete, annotated implementation of the inference function as used in inference.py.

```
import torch
import torch.nn.functional as F
from sentence_transformers import SentenceTransformer

model = SentenceTransformer('paraphrase-multilingual-MiniLM-L12-v2')

# Load all nine anchors once at application startup
ANCHORS = {
'hindi': [torch.load(f'data/baselines/hindi_cluster{i}.pt') for i in range(4)],
'telugu': [torch.load(f'data/baselines/telugu_cluster{i}.pt') for i in range(4)],
'english': [torch.load('data/baselines/english_global.pt')],
}

def quantify(text: str, tau: float = 0.4) -> dict:
"""
Returns a dict: {'hindi': float, 'telugu': float, 'english': float}
Values sum to 100.0.
"""
# Stage 1: Embed input to a unit vector in 384-D space
raw = model.encode([text], normalize_embeddings=True)
v = torch.tensor(raw[0], dtype=torch.float32)

# Stages 2 & 3: Cosine similarity + Contextual Winnowing
scores = {}
for lang, anchors in ANCHORS.items():
# dot product == cosine similarity (both vectors are unit-normalised)
sims = [torch.dot(v, a).item() for a in anchors]
scores[lang] = max(sims) # Winnowing: select best-matching anchor

# Stage 4: Temperature-controlled Softmax -> percentages
s_tensor = torch.tensor(list(scores.values())) / tau
probs = F.softmax(s_tensor, dim=0).tolist()
return {lang: round(p * 100, 1)
for lang, p in zip(scores.keys(), probs)}
```

CHAPTER 9

Configuration and Tuning

9.1 Temperature Parameter τ

The temperature parameter τ is the primary user-facing configuration control. It is exposed as the sidebar slider in the Streamlit application. The table below describes its effect at different values.

τ Value	Behaviour	Dominant Language Score	Recommended Use
0.05	Near-hard classification	approximately 99%	Automated language diarisation
0.10	Hard quantification	approximately 85–90%	Language boundary detection
0.40	Soft density estimation	approximately 50–70%	Sociolinguistic analysis (default)
0.50	Very soft distribution	approximately 45–65%	Research visualisation
1.00	Near-uniform	approximately 35–45%	Exploratory testing only

Table 8: Temperature Parameter Behaviour

9.2 Other Tunable Hyperparameters

Parameter	File	Default	Notes
n_clusters (k)	generate_anchors.py	4	Number of K-Means clusters per language. Higher values give finer
batch_size	generate_anchors.py	64	Sentences per encoding batch. Reduce to 16 or 8 if encountering CU
random_state	generate_anchors.py	42	Random seed for K-Means. Keep at 42 for reproducible anchor files
max_iter (KMeans)	generate_anchors.py	300	Maximum K-Means iterations. Increase only if a ConvergenceWarni
normalize_embeddings	inference.py	True	Must remain True. All distance calculations assume unit-normalised

Table 9: Tunable Hyperparameters

CHAPTER 10

Performance and Evaluation Results

10.1 Evaluation Methodology

The system was evaluated against a dataset of 500 hand-annotated code-mixed sentences spanning the three language pairs. Human annotators were presented with each sentence and asked to estimate the percentage contribution of each language. Mean Absolute Error (MAE) was computed between the annotator consensus and the system output for two methods: a simple token-counting baseline and the vector-based GLQ approach.

10.2 Quantification Accuracy

Language Pair	Token-Counting MAE	GLQ (Vector) MAE	Improvement
Hindi – English	12.4%	4.2%	8.2 percentage points
Telugu – English	14.1%	5.1%	9.0 percentage points
Hindi – Telugu	8.9%	3.8%	5.1 percentage points
Average	11.8%	4.4%	7.4 percentage points

Table 10: Mean Absolute Error Comparison

The largest improvement is observed for Telugu–English pairs (9.0 percentage points), which reflects the structural complexity of Telugu–English code-mixing and its susceptibility to token-counting errors. The vector-based approach handles intra-sentential switching more gracefully because it operates on holistic sentence representations rather than per-token vocabulary matching.

10.3 System Test Cases

The table below documents eight representative test cases used during development. All cases pass at the default $\tau = 0.40$ setting.

TC	Input Sentence	Expected	System Output	Result
01	Mera computer charge nahi ho raha.	Hindi > 80%	Hi 82%, En 15%, Te 3%	Pass
02	The quarterly report ki preparation ho rahi hai.	English + Hindi	En 48%, Hi 46%, Te 6%	Pass
03	Meeting ki preparation cheyyadaniki document ready.	Telugu > 70%	Te 72%, En 24%, Hi 4%	Pass
04	This is a purely English sentence about technology.	English > 90%	En 93%, Hi 4%, Te 3%	Pass
05	Mera naam Bhuvan hai aur main NLP pe kaam karta hoon.	Hindi > 85%	Hi 87%, En 10%, Te 3%	Pass
06	Nenu Telugu lo matladutanu but English also use chestanu.	Telugu + English	Te 44%, En 50%, Hi 6%	Pass
07	Server down ho gaya, database bhi crash kar gaya.	Hindi > 75%	Hi 78%, En 18%, Te 4%	Pass
08	Idi chala important document, please review cheyyi.	Telugu > 75%	Te 76%, En 21%, Hi 3%	Pass

Table 11: Test Cases and Results at τ = 0.40

Test case TC-07 is particularly significant. The sentence *'Server down ho gaya, database bhi crash kar gaya'* contains four English technical terms (*server*, *down*, *database*, *crash*), yet the system correctly classifies it as predominantly Hindi (78%). This directly demonstrates the Contextual Winnowing mechanism: the Hindi-Technical anchor (A4) matches the input more strongly than the English global anchor (A9), because the grammatical connectives and sentence structure are Hindi.

CHAPTER 11

Known Limitations

The following limitations exist in the current implementation and should be understood before extending the system.

Three-Language Constraint

The system supports only Hindi, Telugu, and English. Input containing other Indian languages (Bengali, Tamil, Kannada, etc.) will be incorrectly attributed to one of the three supported languages. Adding new languages is described in Chapter 6.3.

No Romanised Script Support

Romanised Hinglish or Tenglish typed entirely in Roman characters may not embed accurately, as the underlying model's training data contained predominantly native-script text. A transliteration normalisation layer is planned for a future release.

Sentence-Level Granularity Only

The system assigns percentages to the entire sentence, not to individual words or tokens. Per-token language attribution is a planned enhancement (see Chapter 12).

128 Sub-word Token Limit

The SentenceTransformer model truncates input at 128 sub-word tokens (approximately 90–100 words). Long paragraphs should be split into individual sentences before analysis.

Single English Anchor

English is represented by a single global anchor rather than four register-specific anchors. This can cause marginal over-attribution of English in highly technical input text. Adding English register anchors is a low-effort improvement.

No Incremental Anchor Update

Adding new corpus data requires re-running generate_anchors.py on the complete dataset. There is no incremental update mechanism.

CHAPTER 12

Future Enhancements

The following enhancements are identified in priority order. Priority is assessed based on expected impact relative to implementation effort.

Priority	Enhancement	Effort	Description
High	English Register Anchors	Low	Generate four English anchors (narrative, formal, casual, technical) following t
High	Script Normalisation	Medium	Add a transliteration preprocessing layer using the indic-transliteration library s
High	Token-Level Language Attribution	High	Extend inference to produce a per-word language score and render colour-coo
Medium	Additional Indic Languages	Medium	Extend the system to Bengali, Tamil, Kannada, Marathi, and Gujarati using the
Medium	Custom Transformer Fine-Tuning	Very high	Fine-tune the multilingual Sentence-BERT on a code-mixed annotated dataset
Medium	Audio Input via ASR	High	Integrate a code-mixed automatic speech recognition model (e.g., Whisper fin
Low	Real-Time Social Media Streaming	High	Connect to the Twitter/X streaming API to monitor linguistic composition trends
Low	Temporal Linguistic Drift Tracking	Medium	Track and visualise the gradual increase of English in Indian-language social m

Table 12: Future Enhancement Roadmap

CHAPTER 13

References and Datasets

13.1 Key Research Papers

[7,14] N. Reimers and I. Gurevych (2019, 2020)

Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks; and Making Monolingual Sentence Embeddings Multilingual using Knowledge Distillation. These papers describe the embedding model used throughout the GLQ system.

[11] A. Vaswani et al. (2017)

Attention is All You Need. The foundational Transformer architecture paper.

[12] J. Devlin et al. (2019)

BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.

[13] A. Conneau et al. (2020)

Unsupervised Cross-lingual Representation Learning at Scale (XLM-R).

[22] D. Arthur and S. Vassilvitskii (2007)

k-means++: The Advantages of Careful Seeding. Describes the K-Means initialisation strategy used in generate_anchors.py.

[4] M. Rizvi, A. Hussain, and R. Bhatt (2021)

Language Identification in Code-Mixed Indian Social Media Text. Establishes the limitations of token-counting methods addressed by GLQ.

[1] G. Sitaram et al. (2019)

A Survey of Code-switching in Natural Language Processing. Comprehensive background survey on the code-mixing problem.

13.2 Datasets

Dataset	Source	Used For
IndicCorp	AI4Bharat — ai4bharat.org; HuggingFace: ai4bharat/IndicNLPCorpus	Hindi and Telugu sentences for anchor construction (~1.
Wikitext-103	S. Merity et al. (2017) — huggingface.co/datasets/wikitext	English anchor construction (25,000 sampled sentences)
paraphrase-multilingual-MiniLM-L12-v2	HuggingFace: sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2	Pre-trained multilingual sentence embedding model (384
Hand-annotated set	Included in repository: evaluation/annotated_500.csv	System evaluation: 500 code-mixed sentences with hum

Table 13: Datasets and Resources

13.3 Open-Source Libraries

Library	Version	Role in the Project
sentence-transformers	2.x+	SentenceTransformer model wrapper; embedding and model download

PyTorch	2.0+	Tensor operations, anchor serialisation (.pt), Softmax function
scikit-learn	1.3+	K-Means clustering in generate_anchors.py
Streamlit	1.28+	Web application framework; handles UI, routing, and state
Altair	5.x	Declarative interactive charts rendered inside Streamlit
NumPy	1.24+	Numerical array operations; centroid normalisation
Pandas	2.0+	Tabular data handling for the evaluation dataset

Table 14: Open-Source Library Dependencies

Geometric Linguistic Quantifier — Developer Handover Guide
Vukkem Bhuvaneshwar (22211A67C3) | B V Raju Institute of Technology | 2025–2026
Guide: Mr. Krutibash Nayak, Asst. Prof., CSE (DS) | H.O.D.: Dr. R.V. Ramana Chary